

EEE4773 Spring 2026

Fundamentals of Machine Learning

Wake-word Detection

EEE4773 Project: Wake-word Detection

Project Due: April 22, 2026, 11:59 PM

Group Size: up to 3 individuals

Material Due: 4-page final report and code implementation deployed to a GitHub repository

1. Description

In this project, you will develop a machine learning system to detect a specific “wake word” (“Go Gators”) within short audio recordings. Each group will be responsible for collecting audio clips in multiple environments using a phone or laptop microphone. The goal is to build a model that can accurately detect a specific “wake word” within an audio recording based only on sound features.

2. Data Set

Each team must record at least 150 audio clips, each 3-second long. Half of these recordings should contain the designated wake word (“Go Gators”), spoken in varying tones, speeds, and noise environments. The other half should be clips of non-wake words or background noise. Clips should be stored in *.wav* format and labeled accordingly (**1 if wake word is included, or 0 otherwise**). After data collection, all teams’ data will be merged, preprocessed, and split into training and test sets.

In order to standardize data collection as much as possible, please use the Voice Recorder App with the following settings:

- **Microphone adjustment:** Device auto control
- **File format:** .wav
- **Recording quality:** Mono - 48kHz
- **Default file name:** see instructions below
- **Duration:** Trim all your audio clip recordings to have 3 seconds duration exactly (this is important!).

We recommend you to save your files using a coding system, e.g. **ID-trial-label**. First give a number from 1 to 3 to each team member, this is the ID. Then, for example, when team member with ID 2 is recording hers/his/their audio recording with a wake word (label 1), the file name should read "2-3-1.wav".

After collecting all teams' recordings, we will split the data at random into a **training set** and an *easy* **test set**. You will be given the **training set** to train your ML system. We will hold the *easy* **test set** until after you submit your code implementation. This test set will be used for grading, and during your live demo.

We will also create a separate *hard* **test set** that includes readings from different people and environment noises. This test set will be used for **extra credit contest** - see details below.

Once your team collects the data, you should use the Notebook file "EEE4773 Project - Project Data Collection.ipynb" to store your data in a pre-defined format. Follow all the instructions in the Notebook file.

3. Project Report

You should write a report that includes the sections listed below. Your report should follow the IEEE transactions format (single-spaced, double column).

- You can find a (word doc, LaTeX or Overleaf) template for the IEEE transactions format here: <https://www.ieee.org/conferences/publishing/templates.html>

Focus your report on your training and testing strategies for the contest and any unique implementations. The **maximum number** of pages for the report is 4. If there are any pages beyond page 4, they will be discarded and

not read or graded. It should be written with correct English grammar and spelling. Be precise - use pseudo-code or equations to be precise.

For full credit consideration, your report should include the following sections:

- *Abstract.* A summary description of the contents of the report and your findings.
- *Introduction.* Overview of your experiment/s and a literature review. For the literature review, include any references to any relevant papers for your experiment/s. So, whatever you decide to do, search the ACM and IEEE (or other) literature for relevant papers to read and refer to.
- *Implementation.* Describe and outline any specific implementation details for your project. A reader should be able to recreate your implementation and experiments from your project report. If you participate in the extra credit contest, be sure to describe the methodology on how you will identify unknown classes that were not in the training data.
- *Experiments.* Carefully describe your experiments with the training data set and any data augmentation set you constructed or existing data sets. Include a description for the goal of each experiment and experimental findings. This is the bulk of what you will be graded on - if your experimental design is not sound or your experiments do not make sense, you will lose points.
- *Conclusions.* Describe any conclusions or things you learned from the project. Your conclusions must follow from what you did. Do not copy something out of a paper or say something that has no experimental support in the Experiments section.
- *References.* Listing of all references in IEEE bibliography format.

4. Project Code Implementation

You may implement your ML system yourselves or using standard packages/libraries. You can use any packages that come as a default option with Anaconda, TensorFlow, or PyTorch. But, your team must train your ML model from scratch on the provided dataset, i.e. **the use of pre-trained models or transfer learning is not allowed.**

You are welcome to carry any of the following experiments:

- Add more data,
- Apply any data augmentation strategy,
- Design your own model architecture,
- Apply any pre-processing strategy.

Your final code submission should contain 3 files:

- README file - directly edited in your GitHub team repository. A template will be provided.
- train.py or a Notebook (.ipynb) with a function "train". This function should contain the code used to train your final model.
- test.py or a Notebook (.ipynb) with a function "test". This function should load data and labels (in the same format as the training data) and output the predicted labels and a metric score value.
- if you compete in the contest, you can create a separate file for testing on the hard test set (or include it in test.py). This function should receive data and labels (in the same format as the training data) and output the predicted labels and a metric score value.

5. Grading Details

Your grade will be determined using the following rubric:

- 1% Team formation
- 9% Data collection
 - Each team is required to collect 100 audio clips according to the specifications described in Section 2. Grades will be based on data quality, adherence to format requirements, and completeness of the dataset.
- 25% Implementation

- Turn in code that runs correctly and easily on our machines. This requires a very clear README and easy to modify parameter settings. This also requires clearly listing what packages/libraries are needed to run your code - and checking with us before the due date to ensure we have those libraries.
- Turn in code that follows the submission requirements described above.
- 25% Weighted f1-score on "easy" blind test data set
 - Performance will be evaluated using the weighted F1-score on the held-out "easy" blind test set.
 - Your code should generate class predictions for each test audio clip.
 - Full credit will be awarded for achieving a weighted F1-score of ≥ 0.85 , or at least matching the class average, whichever is lower.
- 40% Project report
 - This component will be graded based on the requirements outlined in section 3.

6. Extra Credit Contest

The goal of this project is to implement an end-to-end machine learning system to detect a specific wake word ("Go Gators") . The teams with the best weighted f1-score on the "hard" data set will earn **extra credit**. The "hard" data set will also have 2 labels (wake word present, no wake word present) read by different people and different noise environments. You will want your model to generalize to speaker differences and recording noise.

7. Submission Details

Turn in your project poster and your code on your group GitHub repository by **April 22 at 11:59 PM**. In Canvas, you should submit your GitHub URL.

Be sure your repository contains the following files: train.py, test.py, README.txt, and any saved models that will be needed when demoing your work with the test.py file.